

Progetto d'esame (step 4)

- ATTIVITÀ DI ANALISI: MODELLAZIONE CONCETTUALE
- PRINCIPI BASE DI SOFTWARE DESIGN

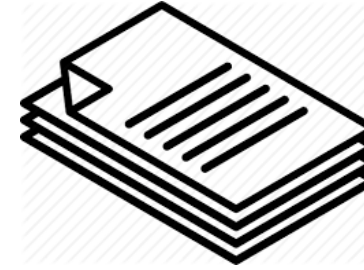
Concetti di progettazione software

Analisi vs Design

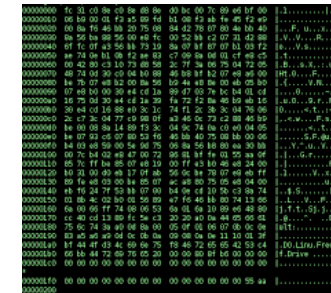
- ❑ Capire l'importanza della fase di Analisi
- ❑ Differenziare Analisi e Design

On the complexity of developing software

- ❑ Input ed output are very/too different ...
- ❑ too complex transformation function

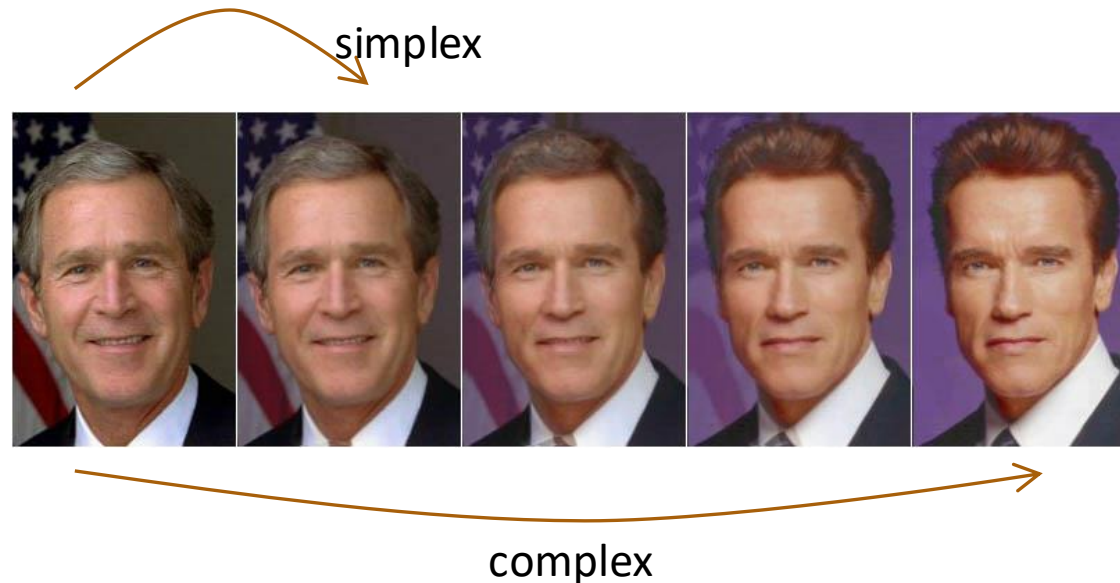


complex transformation



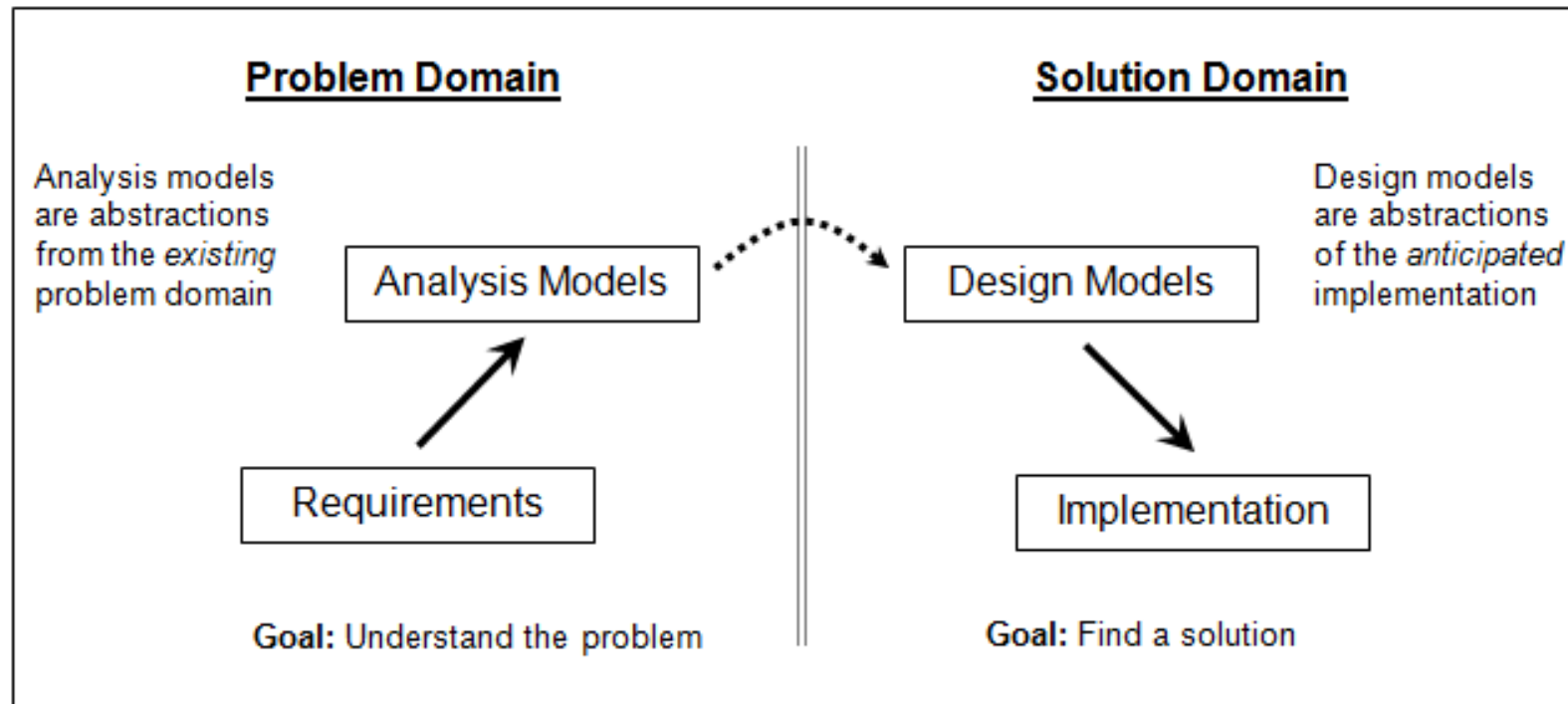
Reducing the complexity

- ❑ Sequence of activities for reducing the complexity of transforming the requirements into code:
 - Requirements → Analysis → Design → Implementation → Testing
- ❑ Each step represents a "simple" transformation function, as in the case of "morphing" ...



Why design is hard

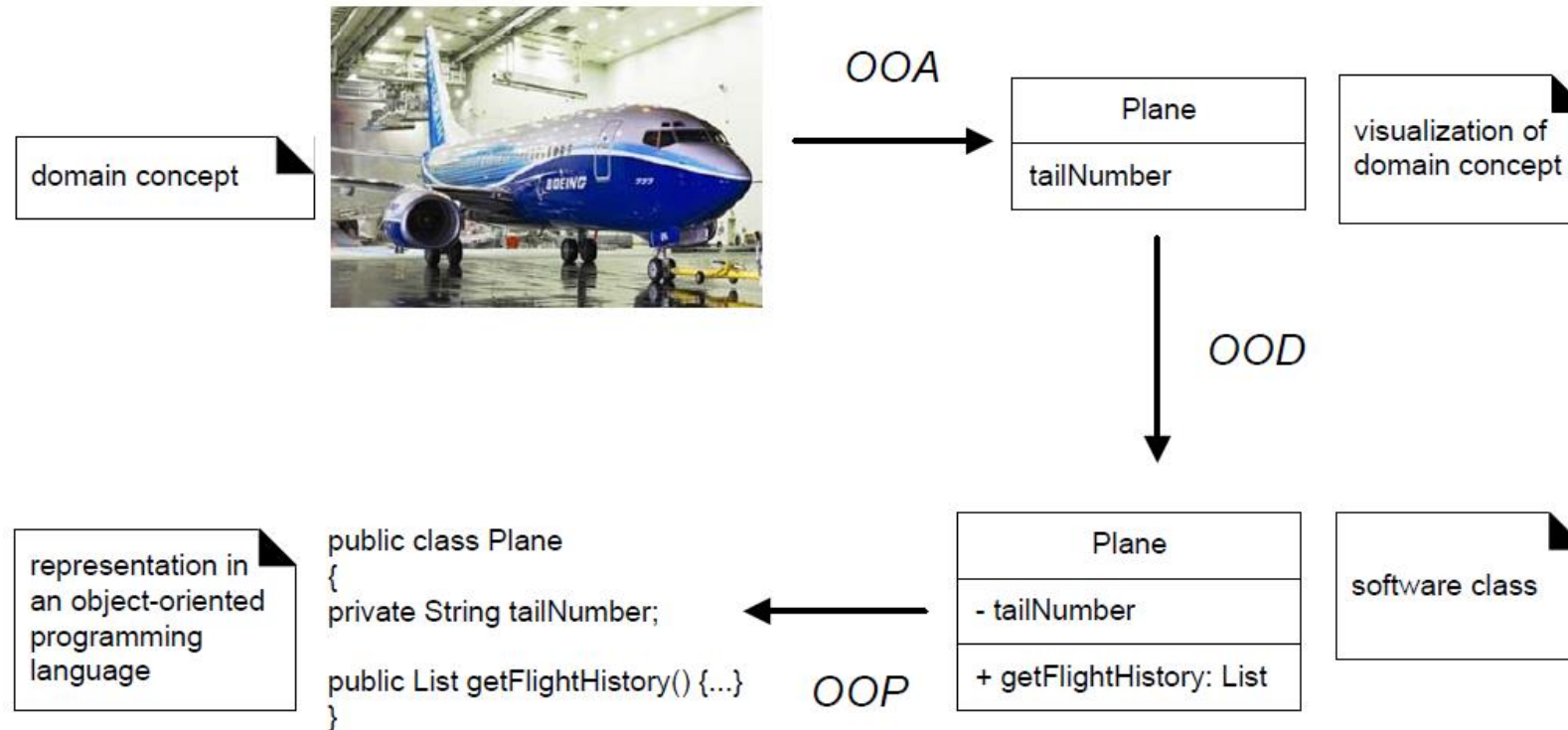
- Design is difficult because design is an abstraction of the solution which has yet to be created



Cosa sono analisi e progettazione OO

- ❑ L'analisi orientata agli oggetti (OOA) enfatizza l'identificazione e la descrizione degli oggetti (**concetti**) nel dominio del problema
 - ad es., nel caso di un sistema informativo per voli, `Plane`, `Flight` e `Pilot`
- ❑ La progettazione orientata agli oggetti (OOD) enfatizza la definizione e la caratterizzazione degli **oggetti software** e delle **loro collaborazioni** al fine di soddisfare i requisiti
 - un oggetto software `Plane` ha un attributo `tailNumber` e un metodo `getFlightHistory`
- ❑ Le classi scelte durante l'OOD vengono implementate durante la programmazione orientata agli oggetti (OOP)
 - una classe `Plane` in Java

Analisi e progettazione OO



L'analisi e la progettazione del software sono attività correlate, così come sono correlate con le altre attività dello sviluppo del software

<step 4>

Analisi (OOA)

- ❑ Modellazione Concettuale (o Modello di Dominio)
 - cenni

Modello di Dominio

- ❑ Passo fondamentale in OOA

- decomposizione di un dominio in **concetti significativi**

- ❑ Modello di Dominio

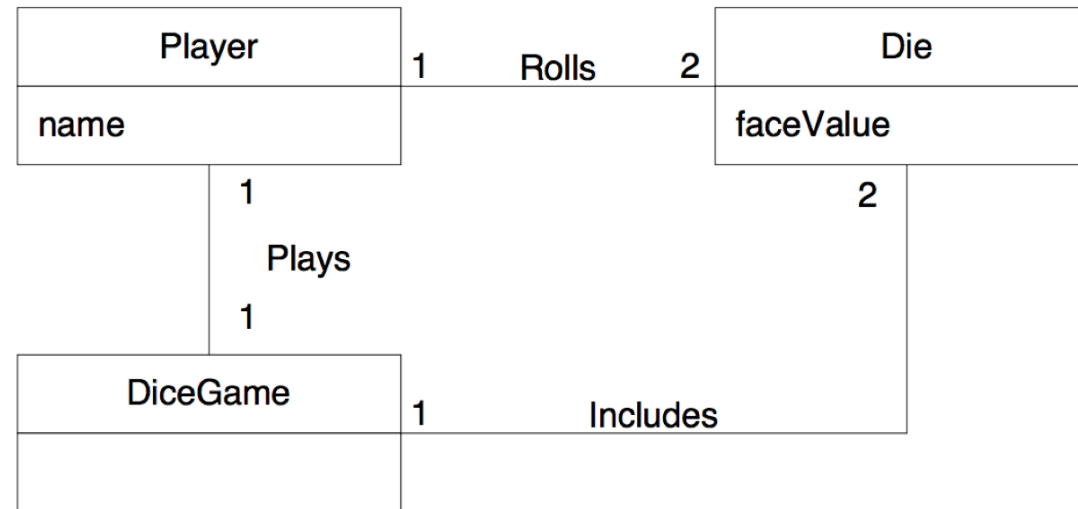
- **rappresentazione visuale** di classi concettuali o di oggetti del mondo reale di un dominio

- ❑ Importante

- rappresentazione di concetti del mondo reale, non componenti software
 - non è un insieme di diagrammi che descrivono classi software, oggetti software e loro responsabilità

Definizione del modello di dominio

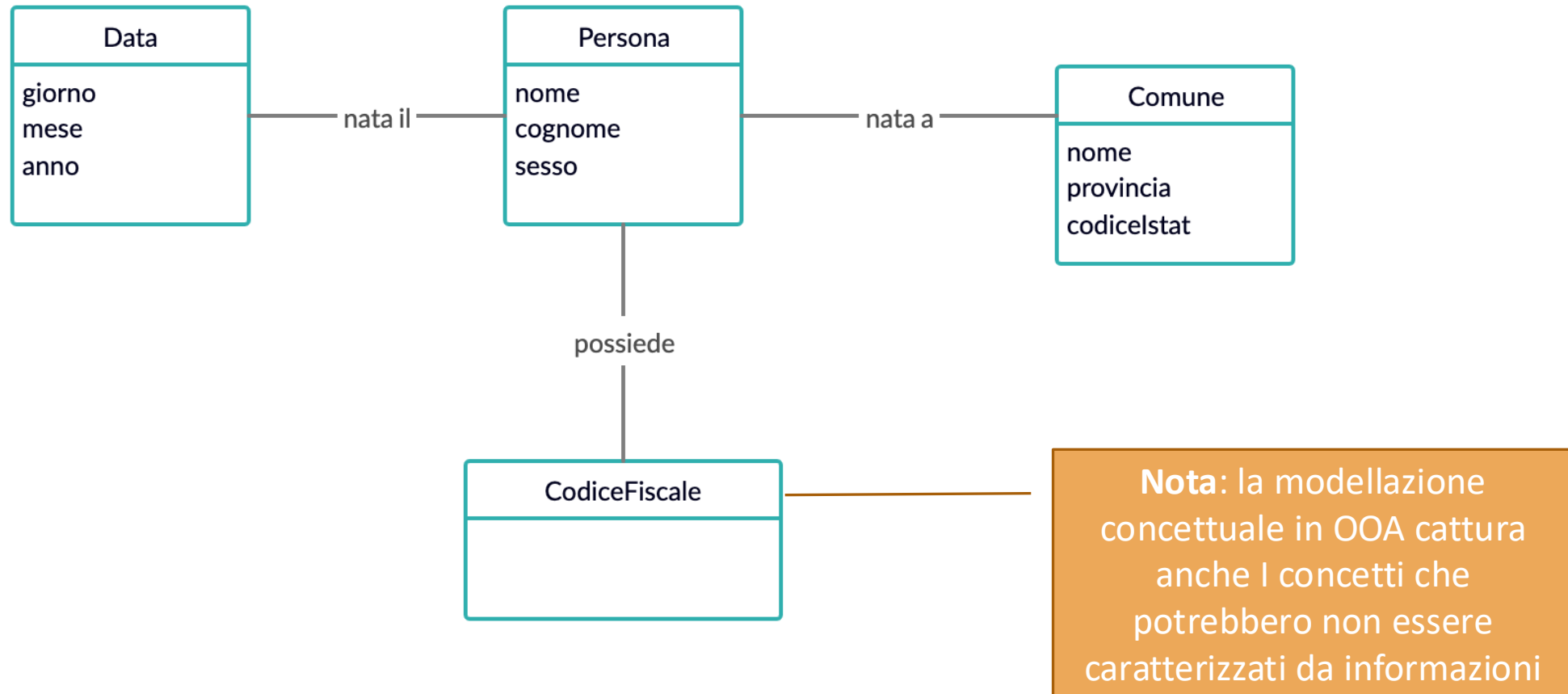
- ❑ L'OOA è interessata a descrivere il dominio di interesse sulla base di una classificazione (concettuale) a oggetti
 - Un **modello di dominio** rappresenta graficamente i concetti, le associazioni e gli attributi significativi del dominio di sistema
 - Attenzione: qui si parla di **oggetti del mondo reale**



Trovare classi concettuali: l'identificazioni di nomi

- ❑ Identificazione di nomi e frasi nominali usando la descrizione testuale del dominio (analisi linguistica)
- ❑ **I casi d'uso dettagliati sono ideali per questo tipo di analisi testuale !!**
- ❑ Non è un mero processo meccanico
- ❑ Alcune parole possono essere ambigue
- ❑ Differenti frasi possono rappresentare lo stesso concetto

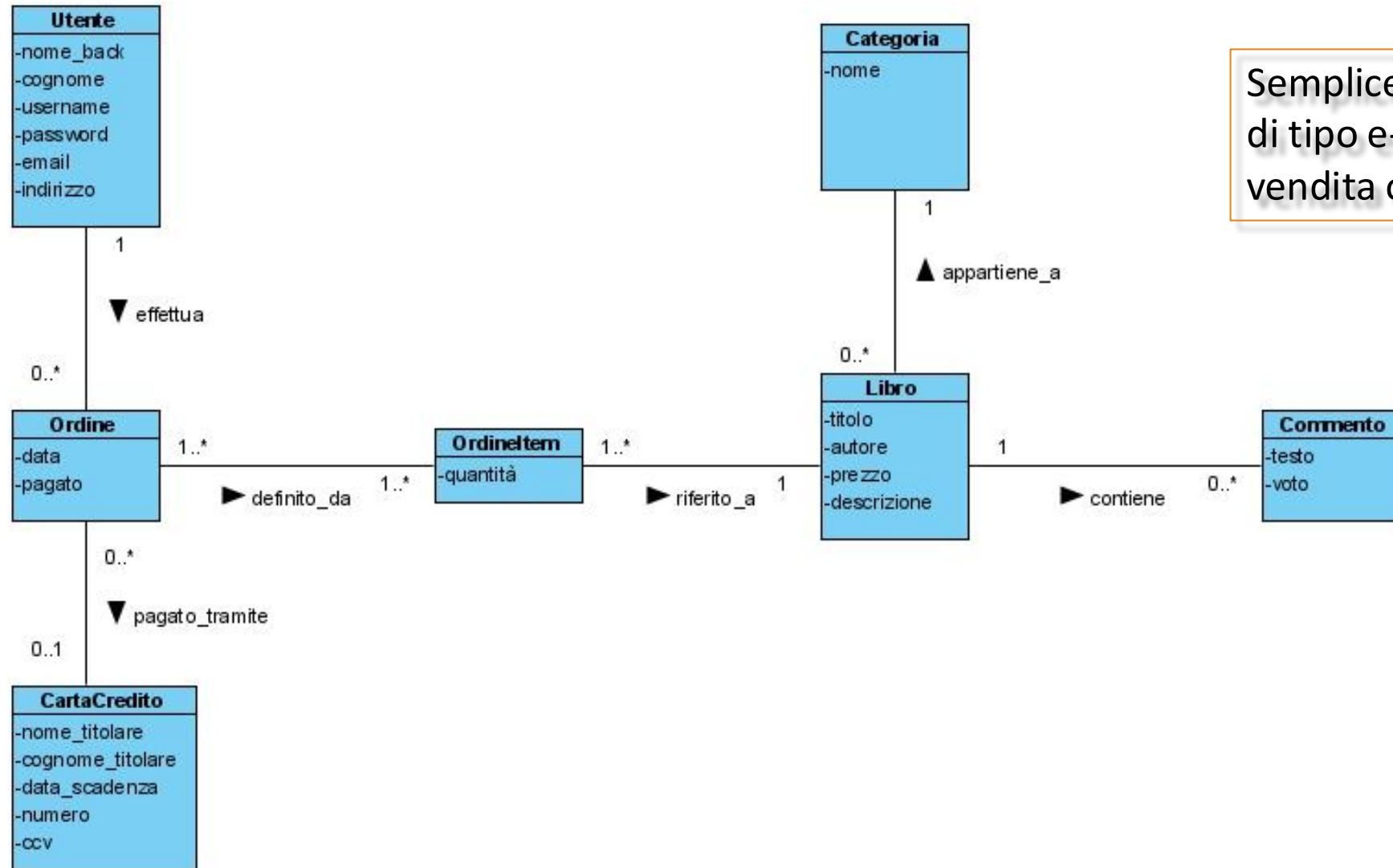
Modello di dominio per “Codice Fiscale”



Come creare il Modello di Dominio

- ❑ Trovare le classi concettuali
- ❑ Disegnare le classi concettuali
 - usare diagramma delle classi (UML)
 - att.ne: abuso di notazione !!
- ❑ Aggiungere associazioni ed attributi

Modello di dominio per “BookStore”



Semplice applicazione web
di tipo e-commerce per la
vendita online di libri

Modello di dominio: linee guida

- ❑ Alcune classi concettuali potrebbero mancare all'inizio della modellazione
- ❑ Puntare ad un “buon” modello iniziale invece che alla perfezione
- ❑ Usare nomi esistenti nel “dominio”: meglio “Passeggero” di “Cliente”
 - Per il calcolo di CF, meglio “comune” che “luogo” in riferimento al luogo di nascita
- ❑ Le associazioni sono aggiunte per maggiore comprensione, non per documentare strutture dati
- ❑ Inoltre:
 - When in doubt if the concept is required, keep the concept
 - When in doubt if the the association is required, drop it
 - Do not keep derivable association

Modello di dominio: linee guida

□ Quando creare nuovi Tipi di Dato?

- Regola: se nel mondo reale un concetto X non è pensato come numero o testo, allora X è probabilmente una classe concettuale (e non un attributo), ma ...
 - è composto da sezioni separate
 - es. phone number, name of person
 - ci sono operazioni associate, come parsing o validazione
 - es. codice fiscale
 - ha altri attributi
 - es. prezzo promozionale con date di inizio e fine
 - è una quantità con unità di misura
 - es. l'importo di un pagamento
 - ...

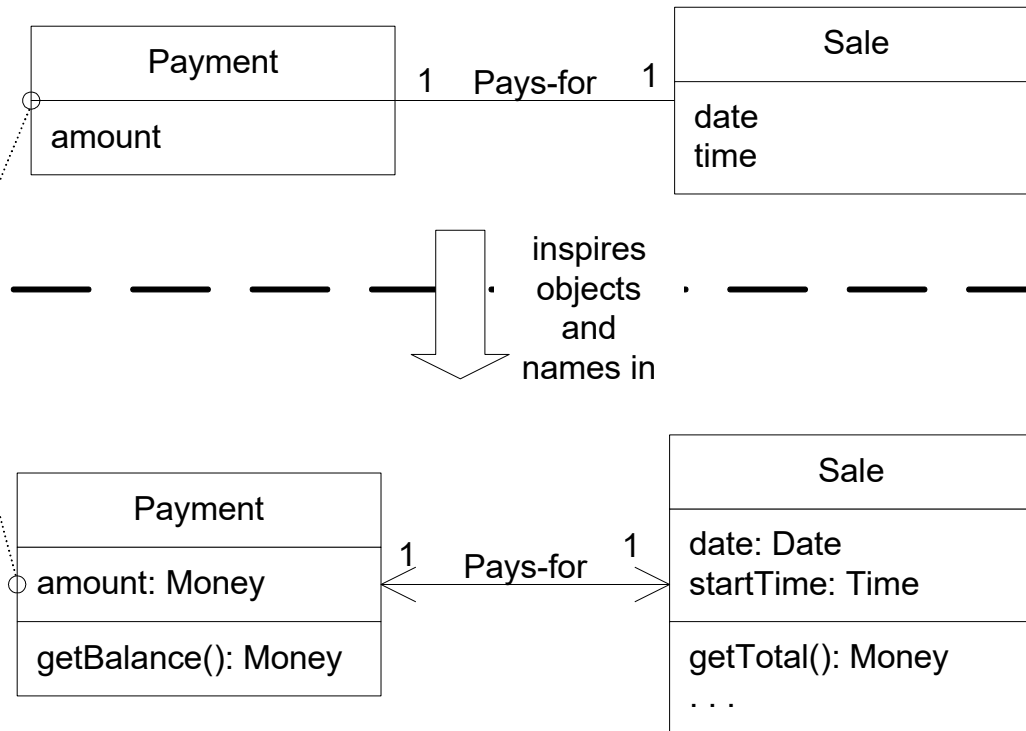
Perché creare un Modello di Dominio

A Payment in the Domain Model is a concept, but a Payment in the Design Model is a software class. They are not the same thing, but the former *inspired* the naming and definition of the latter.

This reduces the representational gap.

This is one of the big ideas in object technology.

UP Domain Model
Stakeholder's view of the noteworthy concepts in the domain.



UP Design Model

The object-oriented developer has taken inspiration from the real world domain in creating software classes.

Therefore, the representational gap between how stakeholders conceive the domain, and its representation in software, has been lowered.

... salto rappresentazionale basso

OOD "semplificato"

- ❑ In OOD coinvolti principi e problematiche profonde
 - l'assegnazione di metodi ha delle conseguenze sulla qualità generale del sistema
 - Principi/problematiche analizzate in dettaglio nel corso di "Ingegneria del Software"
- ❑ In questo corso, "OOD semplificato"
 - "Dopo aver identificato i requisiti ed aver creato un modello di dominio, si aggiungano i metodi alle classi appropriate, e si definiscano i messaggi tra gli oggetti per soddisfare i requisiti"
 - ... ma qualche **principio base** può essere presentato ed adottato

*Concetti di
progettazione
software*

Principi base di software design

- ❑ Principi base di software design
 - Coesione (cohesion)
 - Accoppiamento (coupling)
 - Separazione degli interessi (separation of concerns)
 - Stratificazione (layering)
 - ...

Cohesion and Coupling: two OO Design Principles

- ❑ **Cohesion** and **Coupling** deal with the quality of an OO design
- ❑ Generally, good OO design should be loosely coupled and highly cohesive
- ❑ Lot of the design principles & design patterns which have been created are based on the idea of “Loose coupling and high cohesion”
- ❑ The aim of the design principles should be to make the application:
 - easier to **develop**
 - easier to **maintain**
 - easier to **add new features**

Cohesion

- ❑ **Cohesion** is used to indicate the degree to which a class (in general, a module) has a single, well-focused purpose
 - Higher the cohesiveness of the class, better is the OO design
- ❑ Example:
 - class “Product” with methods for storing/retriving/... data into a database
- ❑ Benefits of Higher Cohesion:
 - highly cohesive classes are much easier to maintain and less frequently changed
 - such classes are more usable than others as they are designed with a well-focused purpose
 - “Single Responsibility Principle” aims at creating highly cohesive classes

Coupling

- ❑ **Coupling** is a measure of how strongly one element is connected to, has knowledge of, or relies on other elements
- ❑ **Low coupling** is an evaluative pattern that dictates how to assign responsibilities to support:
 - lower dependency between the classes
 - change in one class having lower impact on other classes
 - higher reuse potential

Separation of concerns (SoC)

- ❑ SoC is a design principle for separating a computer program into distinct sections, such that each section addresses a separate concern
 - it depends on Cohesion and Low coupling
- ❑ SoC is achieved by encapsulating information inside a section of code that has a well-defined interface
- ❑ **Layered designs** in information systems are another case of separation of concerns (e.g., presentation layer, business logic layer, data access layer, persistence layer)

Progettazione software

Parte II

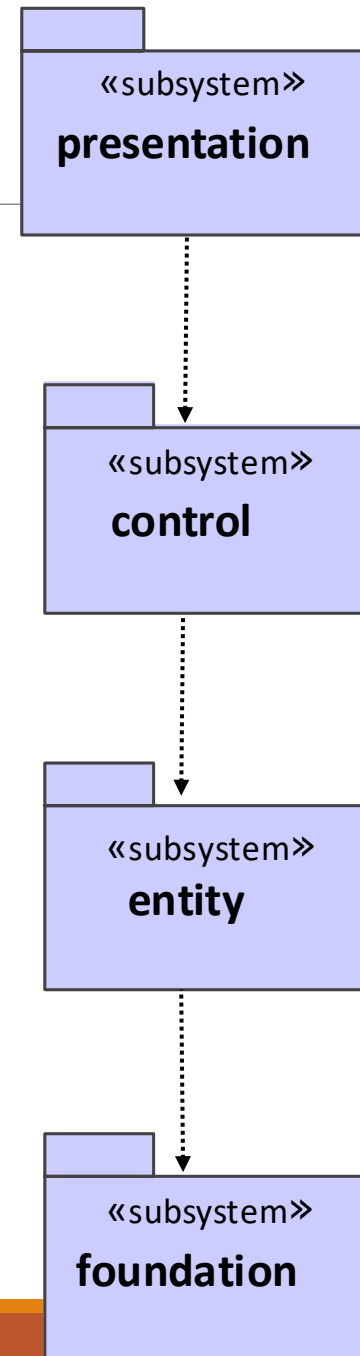
- ❑ Architettura logica di un'applicazione software
- ❑ Layered design ...

Key design principles and guidelines for software architecture

- ❑ Software architecture =
 - structure of a software system
- ❑ System =
 - collection of components that accomplish a specific function or set of functions
- ❑ Software architecture is focused on organizing components to support specific functionalities
- ❑ This organization of functionality is often referred to as grouping components into “areas of concern”
- ❑ ...

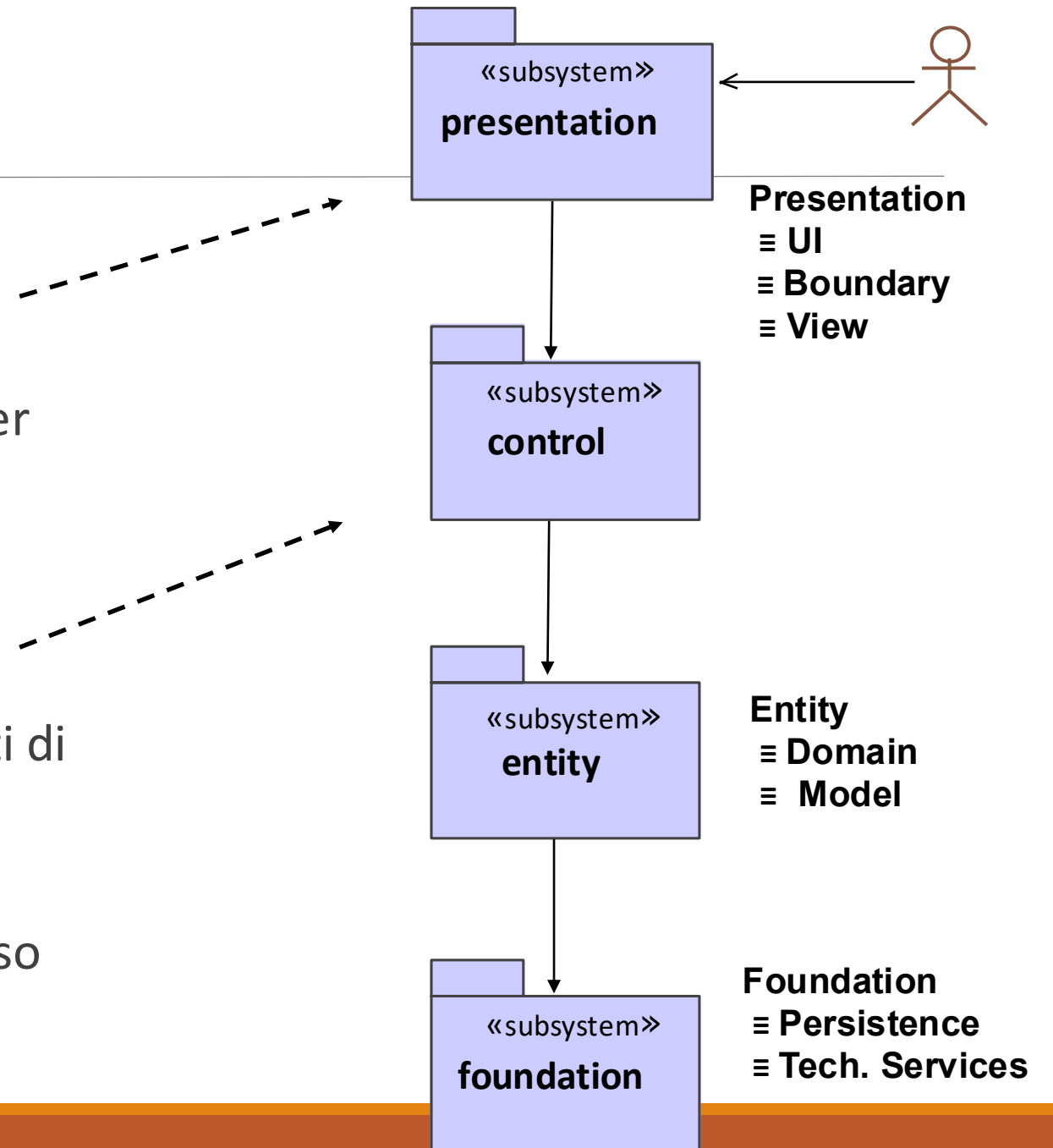
Architettura logica

- ❑ Grouping components into “areas of concerns” ...
- ❑ ... typical organization of sw architecture
- ❑ Known as:
 - PCEF pattern
 - MVC pattern
 - ...



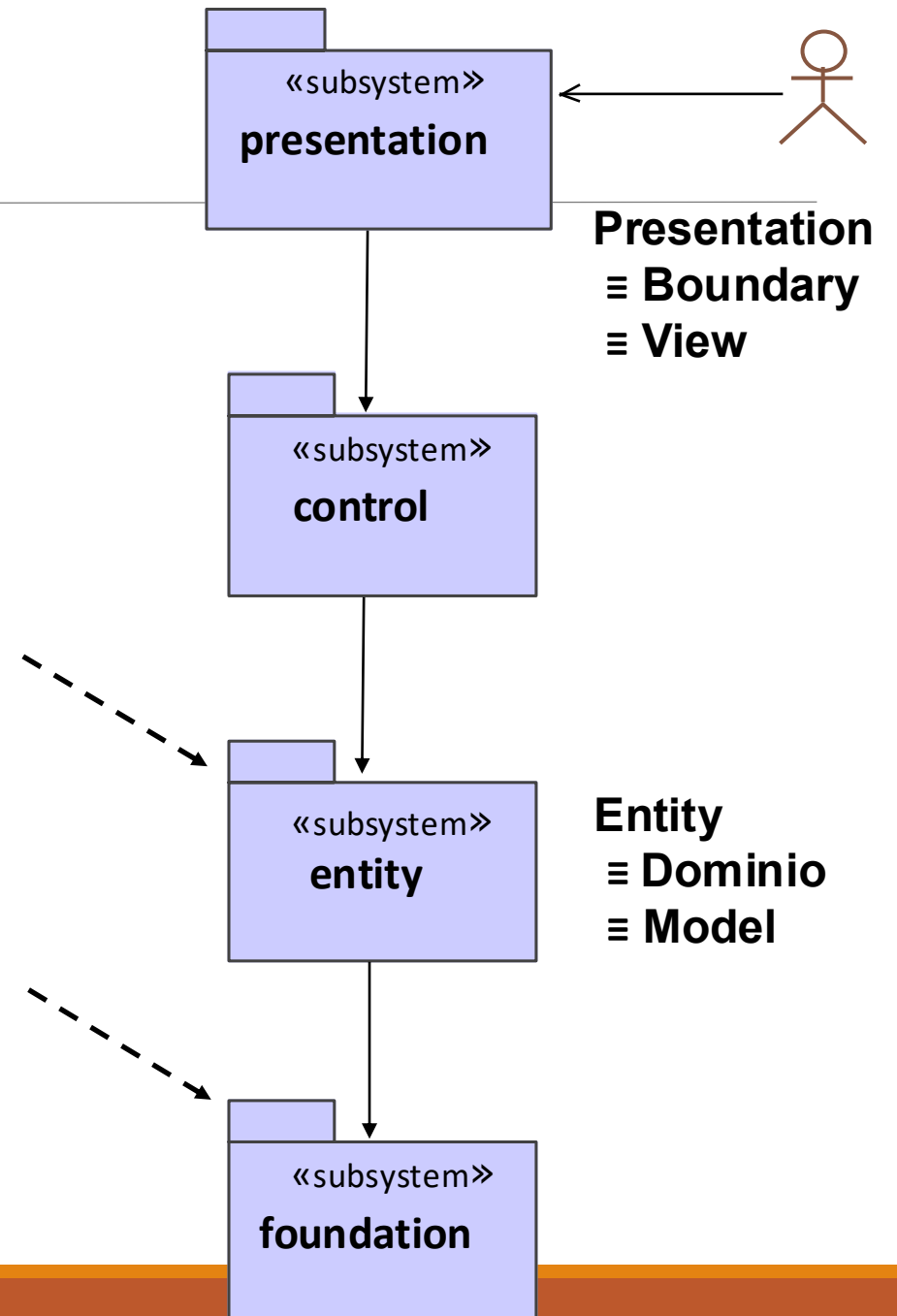
Subsystems (o strati)

- presentation subsystem
 - classi che realizzano la GUI
 - permettono la “human-computer interactions”
- control subsystem
 - classi che disaccoppiano gli strati di presentation e entity
 - conoscono quali classi (entity) implementano un dato caso d’uso

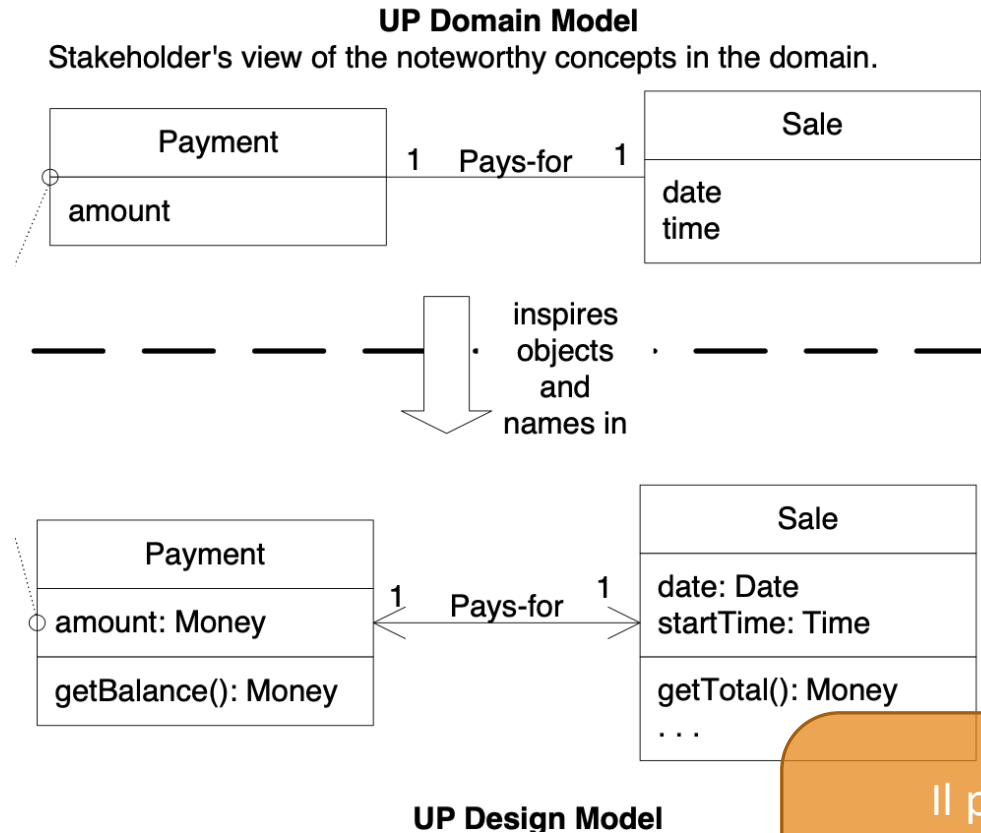


Subsystems

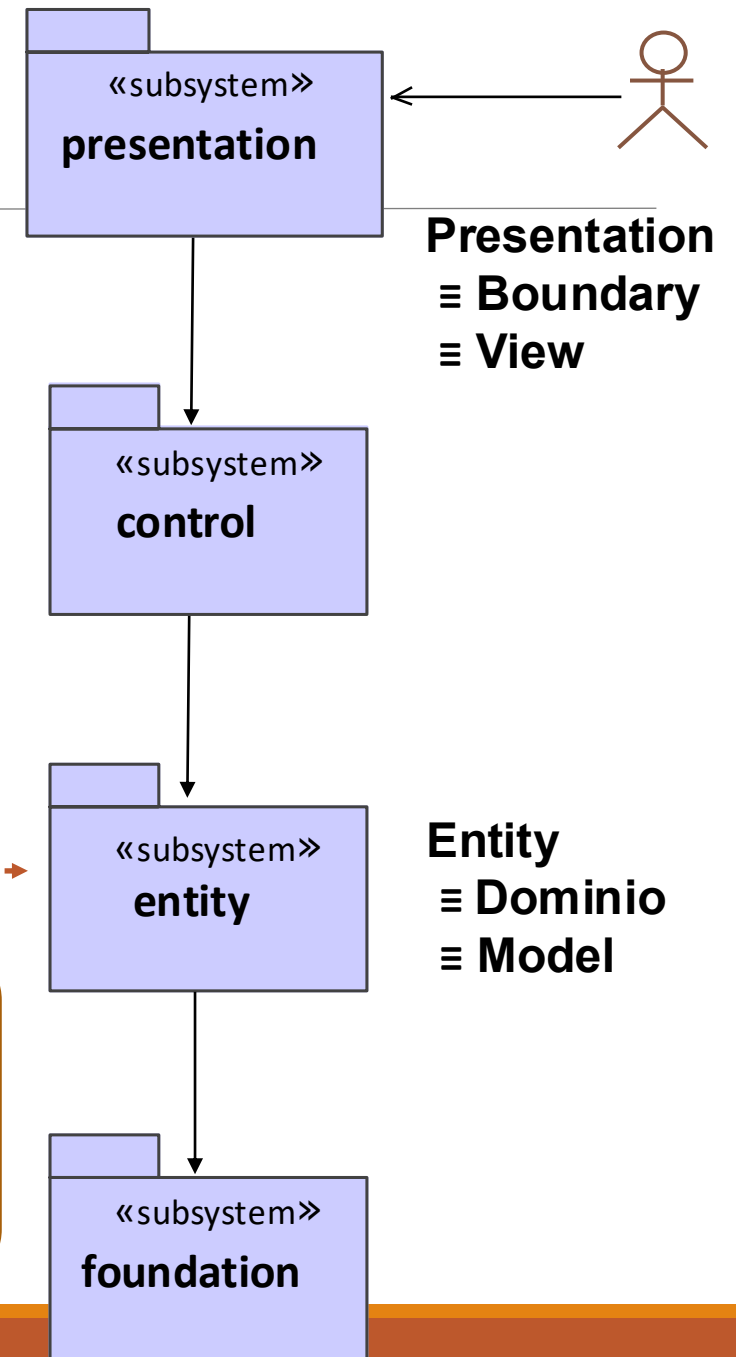
- ❑ Entity subsystem
 - definisce (rete di) oggetti specifici del **dominio applicativo**
 - gestisce oggetti in memoria centrale
 - implementa tutti i requisiti funzionali
- ❑ Foundation subsystem
 - Responsabile dei dati di sessione
 - responsabile della persistenza dei dati
 - conosce l'implementazione a livello fisico dei dati
 - effettua query di accesso al database



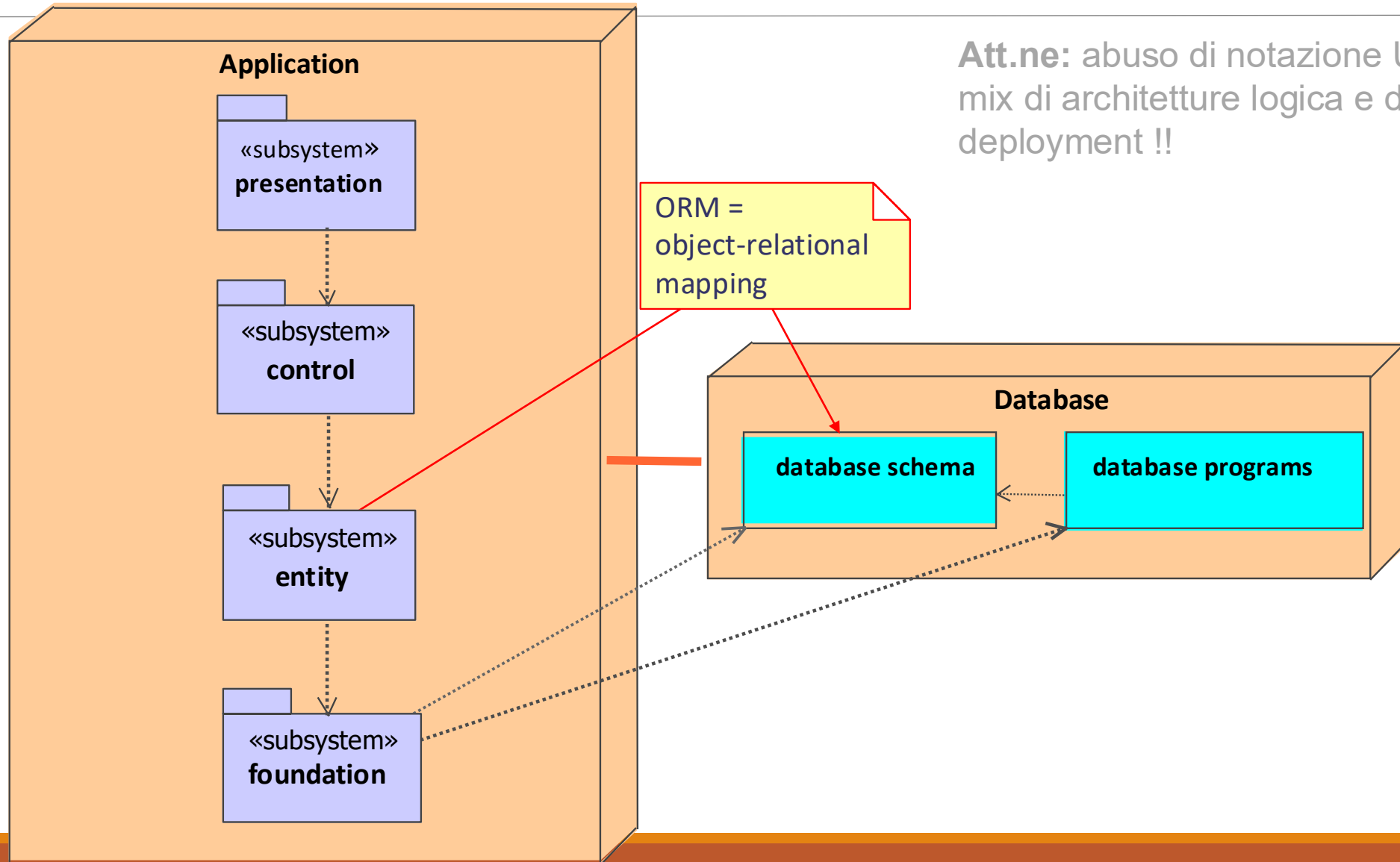
Subsystems



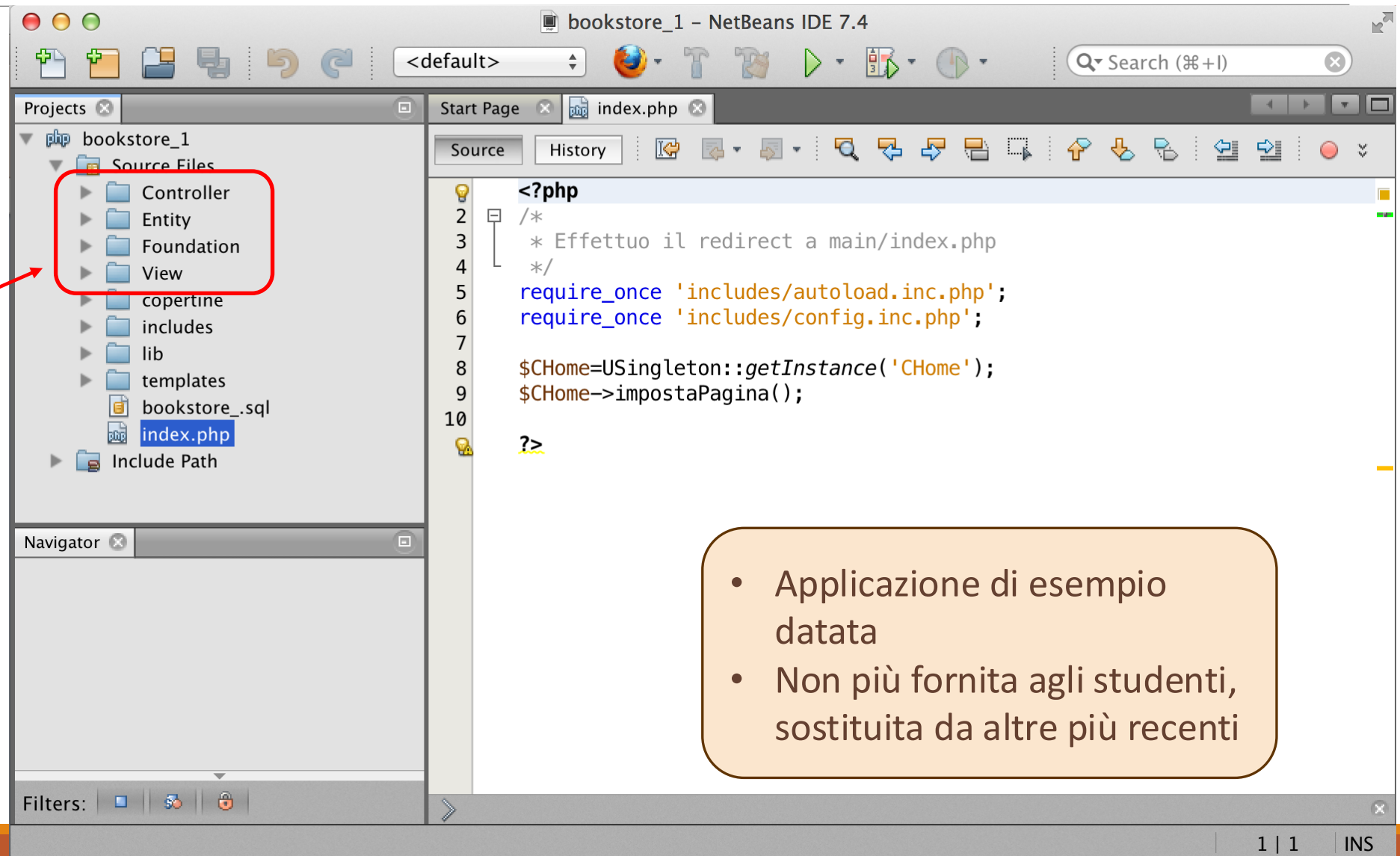
Il package/strato
“entity/model/domain” è
popolato da classi derivate
dal modello concettuale



Integrazione dell'applicazione e progetto DB



Applicazione BookStore



- Strati principali dell'applicazione

- Applicazione di esempio datata
- Non più fornita agli studenti, sostituita da altre più recenti

Applicazione BookStore

